Vault Guardians Protocol Audit Report

Version 1.0

Ynyesto

August 22, 2025

Vault Guardians Protocol Audit Report

Ynyesto

August 2025

Prepared by: Ynyesto

Lead Auditor: Antonio Rodríguez-Ynyesto

Table of Contents

Protocol Summary

Vault Guardians is a DeFi protocol that allows users to deposit ERC20 tokens into ERC4626 vaults managed by fund managers called "vault guardians." The protocol's goal is to maximize yield for depositors through strategic allocation across approved DeFi protocols.

Core Components

- **Vault Guardians**: Human fund managers who can allocate funds between Aave v3, Uniswap v2, or hold positions
- **ERC4626 Vaults**: Standardized vault contracts that manage user deposits and withdrawals
- **Investable Universe**: Limited to Aave v3, Uniswap v2, and holding (no direct transfers allowed)
- **Guardian Staking**: Users must stake a certain amount of ERC20 tokens to become guardians
- **Performance Fees**: Guardians earn fees based on vault performance (documented but not implemented)

Key Features

- **Upgradeable Architecture**: Claims to be upgradeable for platform changes (not implemented)
- **DAO Governance**: Controls pricing parameters and receives performance cuts (partially implemented)
- **Multi-Strategy Support**: Guardians can dynamically adjust allocation ratios between protocols
- **ERC4626 Compliance**: Standard vault interface for deposits, withdrawals, and share management

Technical Architecture

- **VaultGuardians**: Main protocol entry point with DAO controls
- **VaultShares**: Individual vault contracts implementing ERC4626 standard
- **AaveAdapter**: Integration with Aave v3 for lending/borrowing
- **UniswapAdapter**: Integration with Uniswap v2 for liquidity provision
- **Static Token Data**: Abstract contracts managing WETH, USDC, and LINK references

Disclaimer

Ynyesto makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	C	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Scope

The audit covered the following smart contracts:

- **Core Protocol**: `VaultGuardians.sol`, `VaultGuardiansBase.sol`, `VaultShares.sol`
- **Adapters**: `AaveAdapter.sol`, `UniswapAdapter.sol`
- **Abstract Contracts**: `AStaticWethData.sol`, `AStaticUSDCData.sol`, `AStaticTokenData.sol`
- **DAO Components**: `VaultGuardianToken.sol`, `VaultGuardianGovernor.sol`
- **Interfaces**: `IVaultData.sol`, `IVaultGuardians.sol`, `IVaultShares.sol`, `InvestableUniverseAdapter.sol`
- **Vendor Contracts**: `DataTypes.sol`, `IPool.sol`, `IUniswapV2Factory.sol`, `IUniswapV2Router01.sol`

Total Lines of Code: 848 nSLOC

Roles

- **Lead Auditor**: Antonio Rodríguez-Ynyesto
- **Protocol Team**: Vault Guardians Development Team
- **Audit Client**: Vault Guardians Protocol # Executive Summary

Audit Overview

This security audit of the Vault Guardians protocol was conducted by Ynyesto in August of 2025. The audit focused on identifying security vulnerabilities, architectural issues, and implementation flaws in the smart contract codebase.

Key Findings

The audit identified **4 Critical**, **3 High**, **3 Medium**, **8 Low**, **13 Informational** and **2 Gas** findings. The most significant issues include:

Critical Issues

- **False upgradeability claims**: Protocol documentation claims upgradeability without implementing any upgrade mechanism
- **Missing performance fee logic**: Core economic model documented but not implemented
- **Incomplete DAO functionality**: Promised governance features missing from implementation
- **Uniswap slippage vulnerability**: All operations lack slippage protection, enabling MEV exploitation

High Issues

- **WETH vault failure**: Protocol completely broken for WETH as underlying asset
- **Broken ERC4626 accounting**: Missing `totalAssets()` override breaks share pricing
- **Public rebalancing**: MEV surface through publicly callable expensive operations

Risk Assessment

The protocol exhibits several **fundamental architectural flaws** that significantly impact its security and functionality:

1. **Documentation-Implementation Mismatch**: Multiple features documented but not implemented
2. **ERC4626 Non-Compliance**: Core vault standard requirements not met
3. **MEV Vulnerability**: Consistent exposure to sandwich attacks and value extraction
4. **Economic Model Issues**: Fee mechanisms broken, share inflation problems

Recommendations

Immediate Action Required: The protocol should not be deployed to mainnet in its current state. Critical issues require complete architectural redesign before production deployment.

Long-term: Implement proper upgradeability, fix ERC4626 compliance, add comprehensive slippage protection, and align documentation with actual implementation.

Findings

Critical Severity

[C-1] Protocol falsely claims upgradeability without implementing any upgrade mechanism

Description:

The README explicitly states that the protocol is upgradeable:

> "The protocol is upgradeable so that if any of the platforms in the investable universe change, or we want to add more, we can do so."

However, **zero upgradeability mechanisms exist in the codebase**:

- No proxy contracts are inherited by any protocol contracts
- No UUPS, Transparent, or Beacon proxy patterns are implemented
- No upgrade functions or upgrade logic exists
- All contracts are standard, non-upgradeable contracts
- No deployment scripts or configuration indicate upgradeability

Impact:

- **False advertising**: Users are led to believe the protocol can be upgraded when it cannot
- **Security implications**: Users may assume the protocol can fix critical bugs or add new features
- **Protocol limitations**: The protocol cannot adapt to changes in underlying platforms (Aave, Uniswap)
- **User expectations**: Depositors expect a protocol that can evolve and improve over time
- **Documentation deception**: The README makes claims that don't match the actual implementation

Code Location:

```
# README.md
The protocol is upgradeable so that if any of the platforms in the
    investable universe change, or we want to add more, we can do so.
```

```
// src/protocol/VaultGuardians.sol
contract VaultGuardians is Ownable, VaultGuardiansBase {
    //  No proxy inheritance, no upgrade functions
}

// src/protocol/VaultShares.sol
contract VaultShares is ERC4626, IVaultShares, AaveAdapter,
    UniswapAdapter, ReentrancyGuard {
    //  No proxy inheritance, no upgrade functions
}

// src/dao/VaultGuardianToken.sol
contract VaultGuardianToken is ERC20, ERC20Permit, ERC20Votes, Ownable
    {
    //  No proxy inheritance, no upgrade functions
}
```

Root Cause:

The protocol documentation claims upgradeability without implementing any of the standard upgradeability patterns used in the Ethereum ecosystem.

Recommended Mitigation:

Option 1: Implement proper upgradeability (Recommended) This requires significant architectural changes: - Implement UUPS or Transparent proxy pattern - Make core contracts upgradeable through proxy delegation - Add proper upgrade access controls - Implement upgrade safety checks and timelocks

Option 2: Remove false claims If upgradeability cannot be implemented: - Update README to remove false claims about upgradeability - Clarify that the protocol is immutable once deployed - Document the limitations of the current architecture

Critical Note: This is not a simple feature gap - it's a fundamental disconnect between the protocol's stated capabilities and its actual implementation. Users expect an upgradeable protocol based on the documentation, but the current implementation is completely immutable.

[C-2] Complete absence of performance fee logic despite protocol claims

Description:

The protocol documentation and code structure claim that vault guardians charge performance fees, but **zero logic exists to implement this core functionality**:

From README.md: > "Vault guardians charge a performance fee, the better the guardians do, the larger fee they will earn."

From the code: - `GUARDIAN_FEE` constant is defined but never used - `VaultGuardians__UpdatedFee` event is defined but never emitted - No performance calculation logic exists - No fee collection mechanism exists - No fee distribution logic exists

Impact:

- **Protocol deception**: Users are led to believe guardians earn performance fees, but they don't
- **Economic model broken**: The core incentive mechanism for guardians is completely missing
- **Value proposition false**: The protocol cannot deliver on its stated economic model
- **Guardian motivation**: Without performance fees, guardians have no incentive to maximize vault performance
- **User expectations**: Depositors expect guardians to be incentivized to perform well

Code Location:

```
// src/protocol/VaultGuardiansBase.sol
uint256 private constant GUARDIAN_FEE = 0.1 ether; // Defined but NEVER used

// src/protocol/VaultGuardians.sol
event VaultGuardians__UpdatedFee(uint256 oldFee, uint256 newFee); // Defined but NEVER emitted
```

Root Cause:

The protocol appears to be incomplete or abandoned during development. The performance fee system was: 1. **Documented** in the README as a core feature 2. **Partially coded** with constants and events 3. **Never implemented** with actual logic

Recommended Mitigation:

Option 1: Implement complete performance fee system (Recommended) This requires significant development effort to: - Calculate vault performance over time - Implement fee collection mechanisms - Distribute fees to guardians and DAO - Handle fee withdrawal and accounting

Option 2: Remove misleading claims If performance fees cannot be implemented: - Update README to remove false claims about performance fees - Remove unused constants and events - Clarify the actual economic model

Option 3: Implement simplified fee model Instead of performance-based fees, implement: - Fixed percentage fees on deposits/withdrawals - Time-based fees - Flat management fees

Critical Note: This is not a simple bug fix - it's a fundamental missing feature that goes to the heart of the protocol's economic model. The current implementation cannot deliver on its stated value proposition, making this a critical issue that affects the entire protocol's viability.

The absence of performance fee logic suggests the protocol was either never completed or underwent significant scope changes without updating the documentation and removing unused code.

[C-3] DAO lacks core functionality promised in documentation

Description:

The README claims the DAO is responsible for two critical functions:

> "The DAO is responsible for: - Updating pricing parameters - Getting a cut of all performance of all guardians"

However, **the second function does not exist in the protocol**:

1. **"Updating pricing parameters"** - The DAO can update :
    - `s_guardianStakePrice` (stake amount to become guardian)
    - `s_guardianAndDaoCut` (percentage cut from deposits)
2. **"Getting a cut of all performance of all guardians"** - This is completely missing :
    - No performance calculation logic exists
    - No performance fee collection mechanism
    - No way for the DAO to receive performance-based revenue
    - The DAO only gets a fixed cut from deposits (not performance)

Impact:

- **False advertising**: The protocol claims DAO functionality that doesn't exist

- **Economic model broken**: The DAO cannot fulfill its stated purpose
- **Value proposition false**: Users expect DAO governance that isn't implemented
- **Protocol deception**: The documentation misleads users about DAO capabilities
- **Missing revenue stream**: The DAO has no way to earn from guardian performance

Code Location:

```
// README.md claims:
// "The DAO is responsible for:
// - Updating pricing parameters
// - Getting a cut of all performance of all guardians"

// Reality - The DAO can only update these basic parameters:
function updateGuardianStakePrice(uint256 newStakePrice) external
    onlyOwner {
     s_guardianStakePrice = newStakePrice; // Only stake price
}

function updateGuardianAndDaoCut(uint256 newCut) external onlyOwner {
     s_guardianAndDaoCut = newCut; // Only deposit cut
}

// Missing: Performance fee logic, performance calculation, performance
     fee collection
```

Root Cause:

The protocol documentation and implementation are completely misaligned. The DAO was described as having governance over pricing and performance fees, but the implementation only provides basic parameter updates for stake amounts and deposit cuts.

Recommended Mitigation:

Option 1: Implement missing DAO functionality (Recommended) This requires significant development effort to: - Implement performance calculation and tracking - Create performance fee collection mechanisms - Connect the DAO to guardian performance

Option 2: Remove false claims If the DAO functionality cannot be implemented: - Update README to accurately reflect what the DAO actually does - Remove claims about pricing parameter control - Remove claims about performance fee collection - Clarify the actual limited scope of DAO control

Option 3: Implement simplified DAO model Instead of performance-based fees, implement: - Fixed management fees - Time-based fees - Flat protocol fees - Clear governance over basic parameters

Critical Note: This is not a simple feature gap - it's a fundamental disconnect between the protocol's stated value proposition and its actual implementation. The DAO cannot fulfill its documented responsibilities, making this a critical issue that affects the entire protocol's credibility and functionality.

The current implementation suggests the DAO was either never intended to have these powers or the development team abandoned the governance features without updating the documentation.

[C-4] All Uniswap operations lack slippage protection, making them vulnerable to sandwich attacks

Description:

The `UniswapAdapter` contract performs all Uniswap operations with `amountOutMin: 0`, `amountAMin: 0` and `amountBMin: 0`, making every swap, liquidity addition, and liquidity removal vulnerable to sandwich attacks:

1. **`swapExactTokensForTokens`** (line 56): `amountOutMin: 0`
2. **`addLiquidity`** (lines 80-81): `amountAMin: 0, amountBMin: 0`
3. **`removeLiquidity`** (lines 101-102): `amountAMin: 0, amountBMin: 0`

Impact:

- **Sandwich attack vulnerability**: MEV bots can front-run vault operations, manipulate prices, and back-run to extract value
- **Value extraction**: Users lose value on every swap and liquidity operation
- **Economic inefficiency**: The protocol consistently gets worse rates than intended
- **MEV exploitation**: The protocol becomes a target for predatory trading strategies

Code Location:

```
// Lines 56 and 106: Swap without slippage protection
amountOutMin: 0,

// Lines 77-78: Add liquidity without slippage protection
amountAMin: 0,
amountBMin: 0,

// Lines 98-99: Remove liquidity without slippage protection
amountAMin: 0,
amountBMin: 0,
```

Recommended Mitigation:

Implement proper slippage protection by calculating minimum amounts based on expected output and adding a tolerance parameter:

```
// Add slippage tolerance parameter
uint256 public constant SLIPPAGE_TOLERANCE = 50; // 0.5%
```

```
3
4  // Calculate minimum amounts with slippage protection
5  uint256 amountOutMin = (expectedAmountOut * (1000 - SLIPPAGE_TOLERANCE)
       ) / 1000;
6  uint256 amountAMin = (expectedAmountA * (1000 - SLIPPAGE_TOLERANCE)) /
       1000;
7  uint256 amountBMin = (expectedAmountB * (1000 - SLIPPAGE_TOLERANCE)) /
       1000;
8
9  // Use calculated minimums instead of 0
10 amountOutMin: amountOutMin,
11 amountAMin: amountAMin,
12 amountBMin: amountBMin,
```

This would protect users from excessive slippage while maintaining reasonable execution rates.

High Severity

[H-1] The constructor of `VaultShares` tries to get LP tokens from non-existent WETH/WETH Uniswap pools when the asset of the vault is WETH, which breaks the `divestThenInvest` modifier.

Description:

The `VaultShares` constructor attempts to get the Uniswap liquidity token for the `asset + WETH` pair:

```
1  i_uniswapLiquidityToken = IERC20(i_uniswapFactory.getPair(address(
       constructorData.asset), address(i_weth)));
```

However, when the asset is WETH itself, this creates a WETH/WETH pair which doesn't exist in Uniswap. This will cause the constructor to assign the null address to `i_uniswapLiquidityToken`, breaking the `divestThenInvest` modifier and any subsequent Uniswap operations.

Impact:

- WETH vaults would fail during investment/divestment operations, because `i_uniswapLiquidityToken.balanceOf(address(this))` will fail.
- The protocol is fundamentally broken for WETH as an underlying asset

Proof of Concept:

In production, when trying to create a WETH vault:

1. `VaultShares` constructor calls `i_uniswapFactory.getPair(WETH, WETH)`
2. This returns `address(0)` because WETH/WETH pairs don't exist

3. `i_uniswapLiquidityToken` becomes `address(0)`
4. Later calls to `rebalanceFunds()`, `withdraw()` and `redeem()` will fail, because the `divestThenInvest()` will fail when calling `balanceOf()` on the null address.

Recommended Mitigation:

Change the following line in the constructor of `VaultShares`:

```
-       i_uniswapLiquidityToken = IERC20(i_uniswapFactory.getPair(address(constructorData.asset), address(i_weth)));
+       if (address(constructorData.asset) == address(i_weth)) {
+           i_uniswapLiquidityToken = IERC20(i_uniswapFactory.getPair(address(i_weth), address(i_tokenOne)));
+       }
+       else
+       {
+            i_uniswapLiquidityToken = IERC20(i_uniswapFactory.getPair(address(constructorData.asset), address(i_weth)));
+       }
```

[H-2] Incorrect share accounting due to missing `totalAssets()` override

Description:

The `VaultShares.sol` contract inherits from OpenZeppelin's ERC-4626 but does not override `totalAssets()`. The default implementation only considers idle assets held by the vault, ignoring the funds invested into Aave and Uniswap. This creates a fundamental mismatch between the vault's true holdings and its reported TVL.

Impact:

- **Broken share pricing**: `previewDeposit()` and `previewMint()` rely on `_convertToShares()` which uses the incorrect `totalAssets()`. New deposits mint shares in the wrong proportions, breaking fairness between new and existing users.

- **Broken previews**: `previewWithdraw()` and `previewRedeem()` return incorrect results unless the vault forcibly divests beforehand.

- **ERC-4626 non-compliance**: The vault fails to adhere to the standard's requirements for accurate TVL and preview math.

- **Systemic accounting flaw**: Even if patched by workarounds, the accounting model is fundamentally inconsistent with ERC-4626.

Code Location:

```
// src/protocol/VaultShares.sol
contract VaultShares is ERC4626, IVaultShares, AaveAdapter,
    UniswapAdapter, ReentrancyGuard {
     //  Missing totalAssets() override

     // Inherited implementation only counts idle assets:
     // function totalAssets() public view virtual returns (uint256) {
     //     return IERC20(asset()).balanceOf(address(this));
     // }
}
```

Root Cause:

The vault extends ERC-4626 but fails to account for external investments in `totalAssets()`, leaving the accounting incomplete.

Recommended Mitigation:

Implement `totalAssets()` to aggregate balances from Aave, Uniswap LP tokens, and idle funds:

```
function totalAssets() public view override returns (uint256) {
    uint256 idle = IERC20(asset()).balanceOf(address(this));
    uint256 aave = i_aaveAToken.balanceOf(address(this)); // 1:1 with
        underlying
    uint256 lpValueInAsset = calculateLpValueInAsset();
    return idle + aave + lpValueInAsset;
}
```

[H-3] Public `rebalanceFunds()` enables griefing and MEV exploitation when allocations change

Description:

The `rebalanceFunds()` function in `VaultShares.sol` is publicly callable and applies the `divestThenInvest` modifier. This forces the vault to fully unwind its Uniswap LP and Aave positions and then reinvest according to the current allocation data.

At first glance, one might assume this is always exploitable via sandwich attacks since intermediate swaps occur during the rebalance process. However, if the allocation ratios have **not changed**, the vault's round-trip divest/reinvest cycle results in approximately the same state as before (e.g., withdrawing WETH+USDC liquidity, temporarily swapping to balance the pair, then swapping back when re-adding liquidity). In such a case, backrunning the transaction does not yield consistent extractable profit for attackers, since the pool ends up effectively unchanged.

The real vulnerability arises after a vault guardian calls `updateHoldingAllocation`. In that scenario, the next call to `rebalanceFunds()` performs *meaningful allocation changes* — for example, reducing Aave allocation and increasing Uniswap allocation. This requires one-sided trades of predictable direction and large size. These trades are visible in the mempool and can be sandwiched, allowing MEV searchers to extract value from the vault.

Impact:

- **MEV exploitation after allocation changes:** When `updateHoldingAllocation` modifies the strategy, the subsequent `rebalanceFunds` performs large predictable swaps that can be sandwiched for profit at the vault's expense.
- **Griefing risk:** Public callers can repeatedly force full divest/reinvest cycles, exposing the vault to unnecessary slippage and (swap, and liquidity addition and withdrawal) fees even if allocations remain unchanged.
- **Economic inefficiency:** The "divest everything, reinvest everything" approach magnifies costs versus a delta-based rebalance.

Code Location:

```
1 // src/protocol/VaultShares.sol
2 function rebalanceFunds() public isActive divestThenInvest nonReentrant
      {}
3 //  Public access allows anyone to trigger costly operations and expose
      allocation changes to MEV
```

Recommended Mitigation:

1. **Access control:** Restrict `rebalanceFunds()` to trusted roles (e.g., guardian or DAO) rather than leaving it public.
2. **Delta-based rebalancing:** Instead of divesting all funds, compute and execute only the minimal trades required to reach the new allocation.
3. **Slippage checks:** Add strict `minOut` protections to prevent execution at manipulated prices when allocation changes do require swaps.

```
1 // Example guardian-only restriction
2 function rebalanceFunds() public onlyGuardian isActive nonReentrant {
3     _rebalancePortfolio();
4 }
```

Medium Severity

[M-1] The `deposit` function in `VaultShares` mints more shares than assets deposited, creating an inflationary mechanism that dilutes existing shareholders

Description:

In the `deposit` function of `VaultShares`, the protocol mints shares to users based on `previewDeposit(assets)`, but then **additionally** mints shares to the guardian and DAO:

```
uint256 shares = previewDeposit(assets);
_deposit(_msgSender(), receiver, assets, shares);

_mint(i_guardian, shares / i_guardianAndDaoCut);        // 0.1% of shares
_mint(i_vaultGuardians, shares / i_guardianAndDaoCut);  // 0.1% of shares
```

This means that (with the default s_guardianAndDaoCut = 1000) for every deposit: - User gets 100% of the shares they should get - Guardian gets 0.1% extra shares
- DAO gets 0.1% extra shares - **Total: 100.2% of shares are minted per deposit**

Impact:

- **Share dilution**: Existing shareholders see their percentage ownership decrease with each deposit
- **Inflationary mechanism**: The total supply grows faster than the underlying assets
- **Economic imbalance**: Users pay the same amount but the protocol creates additional value for guardians and DAO

Proof of Concept:

The following test demonstrates the vulnerability:

```
    function testMoreSharesMintedThanExpected() public hasGuardian {
        // Initial state: guardian has 100% of shares
        uint256 initialTotalSupply = wethVaultShares.totalSupply();
        uint256 initialGuardianShares = wethVaultShares.balanceOf(guardian);
        uint256 initialVaultGuardiansShares = wethVaultShares.balanceOf(address(vaultGuardians));

        // User deposits assets
        address user = makeAddr("user");
        uint256 depositAmount = 10 ether;
        weth.mint(depositAmount, user);

        vm.startPrank(user);
        weth.approve(address(wethVaultShares), depositAmount);
```

```
14
15          uint256 userSharesBefore = wethVaultShares.balanceOf(user);
16          uint256 guardianSharesBefore = wethVaultShares.balanceOf(
                guardian);
17          uint256 vaultGuardiansSharesBefore = wethVaultShares.balanceOf(
                address(vaultGuardians));
18          uint256 totalSupplyBefore = wethVaultShares.totalSupply();
19
20          uint256 totalExpectedMintedShares = wethVaultShares.
                previewDeposit(depositAmount);
21          uint256 sharesReceivedByUser = wethVaultShares.deposit(
                depositAmount, user);
22
23          vm.stopPrank();
24
25          uint256 guardianAndDaoCut = vaultGuardians.getGuardianAndDaoCut
                ();
26          uint256 userSharesAfter = wethVaultShares.balanceOf(user);
27          uint256 totalSupplyAfter = wethVaultShares.totalSupply();
28          uint256 guardianSharesAfter = wethVaultShares.balanceOf(
                guardian);
29          uint256 vaultGuardiansSharesAfter = wethVaultShares.balanceOf(
                address(vaultGuardians));
30          uint256 sharesActuallyMinted = totalSupplyAfter -
                totalSupplyBefore;
31          uint256 expectedSharesReceivedByUser =
                totalExpectedMintedShares - 2 * (totalExpectedMintedShares /
                 guardianAndDaoCut);
32
33          assertEq(guardianSharesAfter - guardianSharesBefore,
                sharesReceivedByUser / guardianAndDaoCut);
34          assertEq(vaultGuardiansSharesAfter - vaultGuardiansSharesBefore
                , sharesReceivedByUser / guardianAndDaoCut);
35          assertGt(sharesReceivedByUser, expectedSharesReceivedByUser);
36          assertEq(sharesActuallyMinted, sharesReceivedByUser + 2 * (
                sharesReceivedByUser / guardianAndDaoCut));
37          assertGt(sharesActuallyMinted, totalExpectedMintedShares);
38          assertGt(totalSupplyAfter, totalSupplyBefore +
                totalExpectedMintedShares);
39      }
```

Recommended Mitigation:

The guardian and DAO cuts should be taken **from** the user's shares, not **in addition to** them. Modify the `deposit` function:

```
1 uint256 shares = previewDeposit(assets);
2 uint256 guardianCut = shares / i_guardianAndDaoCut;
3 uint256 daoCut = shares / i_guardianAndDaoCut;
4 uint256 userShares = shares - guardianCut - daoCut;
5
```

```
_deposit(_msgSender(), receiver, assets, userShares);
_mint(i_guardian, guardianCut);
_mint(i_vaultGuardians, daoCut);
```

This ensures that exactly 100% of shares are minted per deposit, maintaining the economic balance of the protocol.

[M-2] `sweepErc20s` function in `VaultGuardians` is fundamentally flawed and cannot fulfill its intended purpose

Description:

The `VaultGuardians` contract includes a `sweepErc20s()` function that claims to collect "little bits left around from swapping or rounding errors":

```
/*
 * @notice Any excess ERC20s can be scooped up by the DAO.
 * @notice This is often just little bits left around from swapping or
     rounding errors
 * @dev Since this is owned by the DAO, the funds will always go to the
     DAO.
 * @param asset The ERC20 to sweep
 */
function sweepErc20s(IERC20 asset) external {
    uint256 amount = asset.balanceOf(address(this));
    emit VaultGuardians__SweptTokens(address(asset));
    asset.safeTransfer(owner(), amount);
}
```

However, this function has a critical architectural flaw: **it can only sweep tokens from the `VaultGuardians` contract itself, but user funds and leftover tokens from Uniswap operations are stored in individual `VaultShares` contracts**.

Impact:

- **Function is ineffective**: The sweep function cannot collect the "dust" it's designed to collect
- **Misleading documentation**: The comment suggests it will collect leftover tokens from swaps, but it cannot
- **Wasted gas**: DAO calls to this function will always return 0 (no tokens to sweep)
- **Architectural confusion**: Suggests the developers didn't understand their own protocol design

Root Cause:

The protocol architecture separates concerns incorrectly:

1. **`VaultGuardians`** - Protocol entry point and DAO control (doesn't hold user funds)

2. **`VaultShares`** - Individual vaults that hold user deposits and leftover tokens
3. **`UniswapAdapter`** - Operates within `VaultShares` contracts, leaving dust there

The `holdAllocation` portion of user deposits (which could be 100% of deposits) never leaves the `VaultShares` contract:

```
// src/protocol/VaultShares.sol _investFunds function
function _investFunds(uint256 assets) private {
    uint256 uniswapAllocation = (assets * s_allocationData.
        uniswapAllocation) / ALLOCATION_PRECISION;
    uint256 aaveAllocation = (assets * s_allocationData.aaveAllocation)
         / ALLOCATION_PRECISION;

    _uniswapInvest(IERC20(asset()), uniswapAllocation);
    _aaveInvest(IERC20(asset()), aaveAllocation);
    // holdAllocation is NEVER invested anywhere - it stays in
        VaultShares!
}
```

Code Location:

```
// src/protocol/VaultGuardians.sol
function sweepErc20s(IERC20 asset) external {
    uint256 amount = asset.balanceOf(address(this)); // Only checks
        VaultGuardians balance
    asset.safeTransfer(owner(), amount);
}

// src/protocol/VaultShares.sol
function _investFunds(uint256 assets) private {
    // holdAllocation portion stays in VaultShares as underlying asset
    // Uniswap dust also stays in VaultShares
}
```

Recommended Mitigation:

Option 1: Remove the misleading function entirely (Recommended) Since the function cannot fulfill its intended purpose and any implementation would either be ineffective or dangerous, remove it completely to avoid confusion.

Option 2: Implement safe sweeping with user allocation tracking If sweep functionality is truly needed, the contract must keep track of users' hold allocations to ensure only dust (tokens that exceed user deposits) can be swept:

```
function sweepExcessTokens(IERC20 asset) external onlyOwner {
    uint256 totalUserHoldAllocations = getTotalUserHoldAllocations(
        asset);
    uint256 currentBalance = asset.balanceOf(address(this));

```

```
    require(currentBalance > totalUserHoldAllocations, "No excess
        tokens");
    uint256 excessAmount = currentBalance - totalUserHoldAllocations;

    asset.safeTransfer(owner(), excessAmount);
}
```

This approach requires tracking the total `holdAllocation` amounts in vaults to ensure user funds are never swept.

Note: The existing test `testSweepErc20s()` in `VaultGuardiansTest.t.sol` is misleading because it artificially transfers tokens to the `VaultGuardians` contract, which doesn't happen in normal protocol usage. In reality, user funds are deposited into `VaultShares` contracts by calling the `deposit()` function, making the sweep function ineffective.

The current implementation suggests the developers didn't fully understand their own protocol architecture, making this a significant design flaw that renders the sweep functionality completely ineffective.

[M-3] Inefficient and unsafe reliance on `divestThenInvest`

Description:

Because `totalAssets()` is not correctly implemented, the vault introduces a `divestThenInvest` modifier that divests all funds from Uniswap and Aave before withdrawals/redemptions and reinvests them afterward. This approach is a workaround rather than a proper solution.

Impact:

- **Severe gas inefficiency**: Every redeem/withdraw involves two full portfolio operations (divest + reinvest).

- **Economic leakage**: Fees, slippage, and price impact are incurred by the vault on every withdraw/redeem operation by any user.
- **MEV / griefing risk**: Full divestment exposes the vault to sandwiching on Uniswap operations, as explained in the M-2 finding.

- **Poor UX**: Even small user withdrawals trigger expensive operations affecting all participants.

Code Location:

```
modifier divestThenInvest() {
    if (uniswapLiquidityTokensBalance > 0) {
        _uniswapDivest(IERC20(asset()), uniswapLiquidityTokensBalance);
```

```
    }
    if (aaveAtokensBalance > 0) {
        _aaveDivest(IERC20(asset()), aaveAtokensBalance);
    }
    _;
    if (s_isActive) {
        _investFunds(IERC20(asset()).balanceOf(address(this)));
    }
}
```

Root Cause:

Instead of fixing the accounting issue, the protocol forces full divestment to make `totalAssets()` momentarily accurate, resulting in inefficiency and risk.

Recommended Mitigation:

After implementing a correct `totalAssets()`, replace `divestThenInvest` with targeted withdrawals that divest only what is necessary:

```
function _ensureAssetsAvailable(uint256 assetsNeeded) internal {
    uint256 idle = IERC20(asset()).balanceOf(address(this));
    if (idle >= assetsNeeded) return;

    uint256 shortfall = assetsNeeded - idle;
    uint256 totalInvested = totalAssets() - idle;

    uint256 aaveShare = (i_aaveAToken.balanceOf(address(this)) *
        shortfall) / totalInvested;
    uint256 lpShare = shortfall - aaveShare;

    if (aaveShare > 0) {
        _aaveDivest(IERC20(asset()), aaveShare);
    }
    if (lpShare > 0) {
        uint256 lpTokensToRemove = calculateLpTokensForAssetAmount(
            lpShare);
        _uniswapDivestLpAmount(IERC20(asset()), lpTokensToRemove);
    }
}
```

Notes:

1. Requires a reliable oracle for LP valuation to avoid manipulation.
2. Removes need for full divest/reinvest cycle, saving gas and improving UX.
3. This change depends on implementing a correct `totalAssets()` first.

Low Severity

[L-1] Unnecessary double approval in `UniswapAdapter._uniswapInvest()` function

Description:

The `_uniswapInvest()` function on line 66 approves `amountOfTokenToSwap + amounts[0]`:

```
succ = token.approve(address(i_uniswapRouter), amountOfTokenToSwap + amounts[0]);
```

However, `amounts[0]` represents the amount of input token that was swapped, which equals `amountOfTokenToSwap`. This results in approving `2 * amountOfTokenToSwap`, which is unnecessary and grants to the Uniswap router double the approval needed.

Impact: In the unlikely event that Uniswap were hacked, our vault could be drained, since it approves double what it needs to and then only the right amount of approval is consumed.

Code Location:

```
// Line 66: Double approval
succ = token.approve(address(i_uniswapRouter), amountOfTokenToSwap + amounts[0]);
```

Recommended Mitigation:

Simplify the approval to only approve what's needed:

```
// amounts[0] equals amountOfTokenToSwap, so just approve amountOfTokenToSwap
succ = token.approve(address(i_uniswapRouter), amountOfTokenToSwap);
```

[L-2] Naive liquidity calculation in `UniswapAdapter` makes balanced liquidity addition leave small amounts of the asset in the `UniswapAdapter` contract due to not accounting for price impact, slippage and fees.

Description:

The `_uniswapInvest()` function in `UniswapAdapter` uses a fundamentally flawed approach for adding liquidity:

```
// We will do half in WETH and half in the token
uint256 amountOfTokenToSwap = amount / 2;
```

The function attempts to create a 50/50 liquidity pool by: 1. Swapping half the input token for the counter-party token 2. Adding liquidity with the remaining half + the swapped amount

However, with this approach is mathematically impossible to achieve balanced liquidity because it doesn't account for:

- **Swap fees**: Uniswap charges 1%, 0.3% or 0.05% fees on swaps, reducing the output amount
- **Price impact**: The swap itself moves the price, affecting the optimal liquidity ratio
- **Slippage**: The actual swap output may differ from the expected amount

Impact:

- **Impossible liquidity ratios**: The protocol cannot create truly balanced 50/50 pools
- **Protocol failure**: The core liquidity provision mechanism is fundamentally broken
- **User fund loss**: Depositors receive suboptimal LP positions

Proof of Concept:

Consider adding liquidity to a WETH/USDC pool:

1. User deposits 1000 USDC
2. Protocol swaps 500 USDC for WETH
3. Due to 0.3% fee + price impact + slippage, it receives less than $500 worth of WETH
4. Protocol adds liquidity with: 500 USDC + (WETH received)
5. Result: Some USDC is left in the contract because there wasn't enough WETH to match it.

Code Location:

```
// Line 41: Flawed liquidity calculation
uint256 amountOfTokenToSwap = amount / 2;

// Lines 77-78: Impossible to achieve balanced liquidity
amountADesired: amountOfTokenToSwap + amounts[0],
amountBDesired: amounts[1],
```

Note: This appears to be a known issue in the protocol. The `VaultGuardians` contract includes a `sweepErc20s()` function specifically designed to collect "little bits left around from swapping or rounding errors" and transfer them to the DAO. This suggests the developers were aware that the current liquidity calculation approach would leave leftover tokens in contracts. However, this function is fundamentally flawed because it can only sweep tokens from the `VaultGuardians` contract itself, while user funds and any leftover tokens from Uniswap operations are stored in individual `VaultShares` contracts.

Recommended Mitigation:

Implement proper liquidity calculation that accounts for fees and price impact:

```
function _uniswapInvest(IERC20 tokenIn, uint256 amountIn) internal {
    IERC20 tokenOut = tokenIn == i_weth ? i_tokenOne : i_weth;

    // 1) Get the pair and reserves (ordered per token0/token1)
    address pair = IUniswapV2Factory(i_uniswapRouter.factory()).getPair
        (address(tokenIn), address(tokenOut));
    require(pair != address(0), "No pair");
    (uint112 r0, uint112 r1,) = IUniswapV2Pair(pair).getReserves();

    (uint rIn, uint rOut) =
        address(tokenIn) == IUniswapV2Pair(pair).token0()
            ? (uint(r0), uint(r1))
            : (uint(r1), uint(r0));

    // 2) Compute optimal x to swap from tokenIn -> tokenOut
    uint x = _optimalSwapIn(amountIn, rIn);

    // 3) Swap exactly x with a sane minOut (e.g., 0.5% slippage guard)
    tokenIn.approve(address(i_uniswapRouter), x);
    address[] memory path = new address[](2);
    path[0] = address(tokenIn);
    path[1] = address(tokenOut);

    uint yExpected = UniswapV2Library.getAmountOut(x, rIn, rOut);
    uint yMin = (yExpected * 995) / 1000; // 0.5% slippage example

    uint[] memory amounts = i_uniswapRouter.swapExactTokensForTokens(
        x,
        yMin,
        path,
        address(this),
        block.timestamp
    );

    uint amountTokenA = amountIn - x;   // leftover of tokenIn after
        swap
    uint amountTokenB = amounts[1];     // tokenOut received

    // 4) Add liquidity with non-zero mins (protect vs MEV)
    tokenIn.approve(address(i_uniswapRouter), amountTokenA);
    tokenOut.approve(address(i_uniswapRouter), amountTokenB);

    // Optional: small slippage guard on LP add (e.g., allow 0.5%
        imbalance)
    uint amountAMin = (amountTokenA * 995) / 1000;
    uint amountBMin = (amountTokenB * 995) / 1000;

    (uint aUsed, uint bUsed, uint liq) = i_uniswapRouter.addLiquidity(
        address(tokenIn),
        address(tokenOut),
```

```
48              amountTokenA,
49              amountTokenB,
50              amountAMin,
51              amountBMin,
52              address(this),
53              block.timestamp
54          );
55
56          // 5) Sweep any tiny leftovers if you care (could send back to
                owner/treasury)
57          // uint aLeft = amountTokenA - aUsed;
58          // uint bLeft = amountTokenB - bUsed;
59          // if (aLeft > 0) tokenIn.safeTransfer(treasury, aLeft);
60          // if (bLeft > 0) tokenOut.safeTransfer(treasury, bLeft);
61
62          emit UniswapInvested(aUsed, bUsed, liq);
63      }
64
65      // See Babylonian::sqrt() at https://github.com/Uniswap/solidity-lib/
            blob/master/contracts/libraries/Babylonian.sol
66      function _optimalSwapIn(uint a, uint rIn) internal pure returns (uint)
            {
67          // constants for 0.3% fee (gamma = 0.997)
68          uint numerator = Babylonian.sqrt(rIn * (a * 3988000 + rIn *
                3988009)) - (rIn * 1997);
69          return numerator / 1994;
70      }
```

Key Improvements:

1. **Mathematical precision**: Uses the optimal swap amount formula that accounts for Uniswap's 0.3% fee
2. **Slippage protection**: Implements 0.5% slippage guards on both swap and liquidity addition
3. **Reserve handling**: Correctly calculates reserves considering token ordering
4. **Leftover management**: Can sweep leftover tokens to treasury/owner
5. **MEV protection**: Non-zero minimum amounts prevent sandwich attacks

This approach ensures that the protocol creates the most balanced liquidity pools possible while protecting users from excessive slippage and MEV attacks.

[L-3] Unsafe ERC20 approve operations

Description: The protocol uses `token.approve()` which can lead to issues if the token does not return a boolean, or returns false. It is recommended to use OpenZeppelin's SafeERC20 library and its `safeApprove` function. Some ERC20 tokens do not implement the `approve` function correctly

and do not return a boolean value. Calling them without wrapping them in `SafeERC20` could lead to reverts.

Impact: If a token is used that doesn't follow the ERC20 standard properly, approve calls could fail silently or revert, breaking core functionality.

Code Location:

```
// src/protocol/VaultGuardiansBase.sol Line: 273
bool succ = token.approve(address(tokenVault), s_guardianStakePrice);

// src/protocol/investableUniverseAdapters/AaveAdapter.sol Line: 25
bool succ = asset.approve(address(i_aavePool), amount);

// src/protocol/investableUniverseAdapters/UniswapAdapter.sol Line: 50
bool succ = token.approve(address(i_uniswapRouter), amountOfTokenToSwap
    );

// src/protocol/investableUniverseAdapters/UniswapAdapter.sol Line: 62
succ = counterPartyToken.approve(address(i_uniswapRouter), amounts[1]);

// src/protocol/investableUniverseAdapters/UniswapAdapter.sol Line: 66
succ = token.approve(address(i_uniswapRouter), amountOfTokenToSwap +
    amounts[0]);
```

Recommended Mitigation: Use OpenZeppelin's `SafeERC20` library and `safeApprove` for all `approve` calls.

[L-4] Public functions that could be external

Description: Some public functions are not used internally and could be marked as `external` to save gas. This also improves code clarity by explicitly stating that a function is only meant to be called from outside the contract.

Impact: Minor gas inefficiency and slightly less clear code intent.

Code Location:

```
// src/protocol/VaultShares.sol Line: 115
function setNotActive() public onlyVaultGuardians isActive { ... }

// src/protocol/VaultShares.sol Line: 181
function rebalanceFunds() public isActive divestThenInvest nonReentrant
    {}
```

Recommended Mitigation: Change the visibility of `setNotActive()` and `rebalanceFunds()` from **`public`** to `external`.

[L-5] `nonReentrant` modifier should be placed first

Description: To protect against reentrancy in other modifiers, the `nonReentrant` modifier should be the first modifier in the list. This ensures that the reentrancy guard is active before any other modifier logic is executed, preventing reentrancy attacks that might originate from other modifiers.

Impact: If another modifier on the same function is vulnerable to re-entrancy (e.g., it makes an external call before the `nonReentrant` check), the contract could be exploited. Given that `divestThenInvest` modifier performs external calls, this is a real risk.

Code Location:

```
1  // src/protocol/VaultShares.sol Line: 144
2  function deposit(...) public override(ERC4626, IERC4626) isActive
       nonReentrant { ... }
3
4  // src/protocol/VaultShares.sol Line: 181
5  function rebalanceFunds() public isActive divestThenInvest nonReentrant
        {}
6
7  // src/protocol/VaultShares.sol Line: 193
8  function withdraw(...) public override(ERC4626, IERC4626) isActive
       divestThenInvest nonReentrant returns (uint256) { ... }
9
10 // src/protocol/VaultShares.sol Line: 210
11 function redeem(...) public override(ERC4626, IERC4626) isActive
       divestThenInvest nonReentrant returns (uint256) { ... }
```

Recommended Mitigation: Reorder the modifiers in `deposit`, `rebalanceFunds`, `withdraw`, and `redeem` to place `nonReentrant` as the first modifier.

[L-6] Use of `block.timestamp` as a deadline for Uniswap swaps is insecure

Description: Using `block.timestamp` for swap deadlines offers weak protection. In a Proof-of-Stake context, validators can know in advance if they will propose blocks and can manipulate the timestamp within a certain range. This allows them to withhold a transaction and execute it at a more favorable time.

Impact: A malicious validator could execute a transaction at a time when the price has moved significantly, causing loss of funds for the protocol due to unexpected slippage. This undermines the protection that a deadline is supposed to provide.

Code Location:

```
1  // src/protocol/investableUniverseAdapters/UniswapAdapter.sol Lines 54
       & 104
```

```
// In both swap calls within _uniswapInvest and _uniswapDivest, `block.
    timestamp` is used as the deadline.
i_uniswapRouter.swapExactTokensForTokens({
    // ...
    deadline: block.timestamp
});
```

Recommended Mitigation: Allow the caller to provide a deadline as a parameter for functions that perform swaps. This gives the caller control over how long a transaction can remain pending.

[L-7] Wrong event emitted in `updateGuardianAndDaoCut` function

Description:

The `updateGuardianAndDaoCut` function in `VaultGuardians.sol` emits the wrong event:

```
function updateGuardianAndDaoCut(uint256 newCut) external onlyOwner {
    s_guardianAndDaoCut = newCut;
    emit VaultGuardians__UpdatedStakePrice(s_guardianAndDaoCut, newCut)
        ; // WRONG EVENT!
}
```

The function updates the guardian and DAO cut percentage, but it emits `VaultGuardians__UpdatedStakePrice` instead of the appropriate `VaultGuardians__UpdatedGuardianAndDaoCut` which isn't even defined.

Impact:

- **Misleading events**: Event logs don't accurately reflect what was updated
- **Monitoring confusion**: External systems monitoring events will receive incorrect information
- **Audit complexity**: Event logs don't match the actual function behavior
- **Code inconsistency**: The wrong event name suggests the function updates stake price, not fees

Code Location:

```
// src/protocol/VaultGuardians.sol
event VaultGuardians__UpdatedStakePrice(uint256 oldStakePrice, uint256
    newStakePrice);
event VaultGuardians__UpdatedFee(uint256 oldFee, uint256 newFee);

function updateGuardianAndDaoCut(uint256 newCut) external onlyOwner {
    s_guardianAndDaoCut = newCut;
    emit VaultGuardians__UpdatedStakePrice(s_guardianAndDaoCut, newCut)
        ; // Should be UpdatedFee
}
```

Recommended Mitigation:

Define the correct event and fix the event emission to use it:

```
event VaultGuardians__UpdatedGuardianAndDaoCut(uint256 oldCut, uint256 newCut);

function updateGuardianAndDaoCut(uint256 newCut) external onlyOwner {
    uint256 oldCut = s_guardianAndDaoCut;
    s_guardianAndDaoCut = newCut;
    emit VaultGuardians__UpdatedGuardianAndDaoCut(oldCut, newCut); // Correct event
}
```

[L-8] Unchecked return values from external calls

Description: The return values of `_uniswapDivest`, `_aaveDivest`, and `i_aavePool.withdraw` are not checked. These functions can fail (e.g., if a token transfer fails) or return an amount different from what was expected, but the protocol does not handle these cases.

Impact: If one of these external calls fails or behaves unexpectedly, the protocol could end up in an inconsistent state, potentially leading to a loss of funds. For example, if a divest operation fails silently, the protocol might proceed as if the funds were available, leading to failed transactions or incorrect accounting.

Code Location:

```
// src/protocol/VaultShares.sol Line: 74 in divestThenInvest modifier
_uniswapDivest(IERC20(asset()), uniswapLiquidityTokensBalance);

// src/protocol/VaultShares.sol Line: 77 in divestThenInvest modifier
_aaveDivest(IERC20(asset()), aaveAtokensBalance);

// src/protocol/investableUniverseAdapters/AaveAdapter.sol Line: 44 in _aaveDivest
i_aavePool.withdraw({ ... });
```

Recommended Mitigation: Check the return values of these functions. For `withdraw`, ensure the amount returned is as expected. For `_uniswapDivest` and `_aaveDivest`, their return values should be checked in the `divestThenInvest` modifier to ensure the divestment was successful before proceeding.

Informational Findings

[I-1] Incorrect comment in `UniswapAdapter._uniswapInvest()` function

Description:

The following comment in the `UniswapAdapter::_uniswapInvest()` function is incorrect:

```
* @notice So we swap out half of the vault's underlying asset token for
    WETH if the asset token is USDC or WETH
```

This should read "if the asset token is USDC or LINK (or any other token)" since WETH is handled as a special case in the logic. When the asset is WETH, the function swaps half of it for USDC (`i_tokenOne`), not for WETH.

Impact:

- **Documentation confusion**: Developers may misunderstand the intended behavior
- **Code maintainability**: Misleading comments make the code harder to understand and maintain

Code Location:

```
// Line 32: Incorrect comment
* @notice So we swap out half of the vault's underlying asset token for
    WETH if the asset token is USDC or WETH
```

Recommended Mitigation:

Fix the comment to accurately reflect the logic:

```
* @notice So we swap out half of the vault's underlying asset token for
    WETH if the asset token is USDC or LINK
```

[I-2] Misleading comment about `amounts[1]` in `UniswapAdapter._uniswapInvest()` function

Description:

The comment on line 73 states:

```
// amounts[1] should be the WETH amount we got back
```

This is misleading because `amounts[1]` is only the WETH amount when swapping from a non-WETH token to WETH. When the asset is WETH itself, `amounts[1]` represents the USDC amount received from swapping WETH to USDC.

Impact:

- **Code confusion**: Developers may misunderstand what `amounts[1]` represents
- **Maintenance issues**: Incorrect comments make future code modifications more error-prone

Code Location:

```
// Line 71: Misleading comment
// amounts[1] should be the WETH amount we got back
```

Recommended Mitigation:

Update the comment to be more accurate:

```
// amounts[1] is the amount of counterPartyToken received from the swap
```

[I-3] Empty and unused interfaces create confusion and suggest incomplete protocol design

Description:

The protocol contains several empty interfaces that are not used anywhere in the codebase:

1. `IVaultGuardians` (src/interfaces/IVaultGuardians.sol): Completely empty interface with no function signatures
2. `IInvestableUniverseAdapter` (src/interfaces/InvestableUniverseAdapter.sol): Interface with both function signatures commented out

These interfaces suggest that the protocol was designed with a specific architecture in mind but was never fully implemented, creating confusion for developers and auditors.

Impact:

- **Developer confusion**: Empty interfaces suggest incomplete or abandoned design patterns
- **Code maintainability**: Unused interfaces make the codebase harder to understand
- **Audit complexity**: Auditors must determine whether these are intentional or oversight
- **Protocol clarity**: Suggests the protocol design may have evolved from the original architecture

Code Location:

```
// src/interfaces/IVaultGuardians.sol
interface IVaultGuardians {} // q why empty?

// src/interfaces/InvestableUniverseAdapter.sol
interface IInvestableUniverseAdapter { // q why are both functions
    commented out?
// function invest(IERC20 token, uint256 amount) external;
// function divest(IERC20 token, uint256 amount) external;
}
```

Recommended Mitigation:

Either: 1. **Remove unused interfaces** if they're not needed 2. **Implement the interfaces** if they represent intended functionality 3. **Add clear documentation** explaining why they exist but are empty

If these interfaces are meant for future use, add TODO comments explaining their intended purpose and timeline for implementation.

[I-4] Confusing variable naming in `AStaticUSDCData.sol` makes the code harder to understand

Description:

The `AStaticUSDCData.sol` contract uses generic variable and constant names when it's specifically designed to work with USDC:

```
// Intended to be USDC
IERC20 internal immutable i_tokenOne;
string public constant TOKEN_ONE_VAULT_NAME = "Vault Guardian USDC";
string public constant TOKEN_ONE_VAULT_SYMBOL = "vgUSDC";
```

However, throughout the codebase, these variables are consistently used as if they were USDC, and the constructor parameter is even named `usdc` in some places. The generic names `i_tokenOne`, `TOKEN_ONE_VAULT_NAME`, and `TOKEN_ONE_VAULT_SYMBOL` are misleading and don't reflect the actual intended behavior.

Impact:

- **Code readability**: Developers may not immediately understand that `i_tokenOne` is USDC
- **Maintenance confusion**: Future developers might think this variable can be any token
- **Audit complexity**: Auditors must trace through the code to understand the actual usage
- **Inconsistent naming**: The variable name doesn't match its intended purpose

Code Location:

```
// src/abstract/AStaticUSDCData.sol
// Intended to be USDC
IERC20 internal immutable i_tokenOne;
string public constant TOKEN_ONE_VAULT_NAME = "Vault Guardian USDC";
string public constant TOKEN_ONE_VAULT_SYMBOL = "vgUSDC";

constructor(address weth, address tokenOne) AStaticWethData(weth) {
    i_tokenOne = IERC20(tokenOne);
}
```

Recommended Mitigation:

Rename the variable and constants to clearly indicate their purpose:

```
// Change from generic to specific
IERC20 internal immutable i_usdc;
```

```
3  string public constant USDC_VAULT_NAME = "Vault Guardian USDC";
4  string public constant USDC_VAULT_SYMBOL = "vgUSDC";
5
6  constructor(address weth, address usdc) AStaticWethData(weth) {
7      i_usdc = IERC20(usdc);
8  }
9
10 function getUsdc() external view returns (IERC20) {
11     return i_usdc;
12 }
```

This makes the code more self-documenting and eliminates confusion about the variable's intended purpose.

[I-5] Misleading comment in `AStaticWethData.sol` references non-existent tokens

Description:

The `AStaticWethData.sol` contract contains a confusing comment that suggests it defines multiple tokens:

```
1 // The following four tokens are the approved tokens the protocol
      accepts
2 // The default values are for Mainnet
```

However, this contract only defines one token (`i_weth`) and its associated constants. The comment about "four tokens" is misleading and doesn't match the actual implementation.

Impact:

- **Developer confusion**: The comment suggests there should be four tokens defined in this contract
- **Code inconsistency**: The comment doesn't match the actual code structure
- **Maintenance issues**: Future developers might look for missing token definitions
- **Audit complexity**: Auditors must determine if tokens are missing or if the comment is wrong

Code Location:

```
1 // src/abstract/AStaticWethData.sol
2 // The following four tokens are the approved tokens the protocol
      accepts
3 // The default values are for Mainnet
4 IERC20 internal immutable i_weth;
5 // slither-disable-next-line unused-state
6 string internal constant WETH_VAULT_NAME = "Vault Guardian WETH";
7 // slither-disable-next-line unused-state
8 string internal constant WETH_VAULT_SYMBOL = "vgWETH";
```

Recommended Mitigation:

Update the comment to accurately reflect what the contract actually defines:

```
// This contract defines the WETH token and its associated vault
    metadata
// The default values are for Mainnet
IERC20 internal immutable i_weth;
```

Alternatively, if the comment refers to the entire inheritance chain (WETH + USDC + LINK), clarify this:

```
// This contract defines the WETH token. Combined with child contracts,
// the protocol accepts WETH, USDC, and LINK as approved tokens.
// The default values are for Mainnet
```

This eliminates confusion and makes the code more maintainable.

[I-6] Hardcoded LINK values in `AStaticTokenData.sol` contradict the contract's generic design

Description:

The `AStaticTokenData.sol` contract is designed to be generic and work with any token, but it contains hardcoded LINK-specific values that contradict this design:

```
// Intended to be LINK
IERC20 internal immutable i_tokenTwo;
string public constant TOKEN_TWO_VAULT_NAME = "Vault Guardian LINK";
string public constant TOKEN_TWO_VAULT_SYMBOL = "vgLINK";
```

The contract's generic design allows it to work with LINK or any other tokens that may be added in the future, but the hardcoded vault name and symbol are specifically tied to LINK. This creates a contradiction between the contract's intended flexibility and its actual implementation.

Impact:

- **Design inconsistency**: The contract claims to be generic but has token-specific hardcoded values
- **Limited flexibility**: Adding new tokens requires code changes instead of just constructor parameters
- **Maintenance overhead**: Future token additions require editing the hardcoded names.
- **Architectural confusion**: The contract structure suggests flexibility but the implementation is rigid

Code Location:

```
// src/abstract/AStaticTokenData.sol
// Intended to be LINK
IERC20 internal immutable i_tokenTwo;
string public constant TOKEN_TWO_VAULT_NAME = "Vault Guardian LINK";
string public constant TOKEN_TWO_VAULT_SYMBOL = "vgLINK";

constructor(address weth, address tokenOne, address tokenTwo)
    AStaticUSDCData(weth, tokenOne) {
     i_tokenTwo = IERC20(tokenTwo);
}
```

Recommended Mitigation:

Make the vault name and symbol configurable through the constructor to maintain the contract's generic design:

```
abstract contract AStaticTokenData is AStaticUSDCData {
    IERC20 internal immutable i_tokenTwo;
    string public immutable i_tokenTwoVaultName;
    string public immutable i_tokenTwoVaultSymbol;

    constructor(
        address weth,
        address tokenOne,
        address tokenTwo,
        string memory vaultName,
        string memory vaultSymbol
    ) AStaticUSDCData(weth, tokenOne) {
        i_tokenTwo = IERC20(tokenTwo);
        i_tokenTwoVaultName = vaultName;
        i_tokenTwoVaultSymbol = vaultSymbol;
    }

    function getTokenTwo() external view returns (IERC20) {
        return i_tokenTwo;
    }
}
```

This approach maintains the contract's generic nature while allowing each deployment to specify appropriate vault metadata for whatever token is being used.

[I-7] Confusing inheritance hierarchy for static token data

Description:

The protocol defines static token references through a chain of abstract contracts:

```
// Chain 1: VaultGuardiansBase
```

```
2 AStaticWethData -> AStaticUSDCData -> AStaticTokenData ->
      VaultGuardiansBase
3
4 // Chain 2: UniswapAdapter
5 AStaticWethData -> AStaticUSDCData -> UniswapAdapter
```

This creates two different inheritance paths for similar functionality. - **VaultGuardiansBase** inherits from `AStaticTokenData` and therefore has access to: - `i_weth` (from AStaticWethData) - `i_tokenOne` (from AStaticUSDCData) - `i_tokenTwo` (from AStaticTokenData) - **UniswapAdapter** inherits only from `AStaticUSDCData`, so it only has access to: - `i_weth` - `i_tokenOne`

Impact:

- **Architectural inconsistency**: Different parts of the system expose different subsets of tokens despite similar naming and structure.
- **Maintainability risk**: Adding or changing token definitions requires reasoning about which contracts "see" which tokens.
- **Onboarding confusion**: The inheritance chain suggests all contracts share a consistent token model, but they don't.

This is not a direct security risk, but it increases the complexity of understanding and maintaining the system.

Code Location:

```
1 // src/protocol/investableUniverseAdapters/UniswapAdapter.sol
2 contract UniswapAdapter is AStaticUSDCData { // Only gets i_weth +
      i_tokenOne
3
4 // src/protocol/VaultGuardiansBase.sol
5 contract VaultGuardiansBase is AStaticTokenData, IVaultData { // Gets
      i_weth + i_tokenOne + i_tokenTwo
```

Recommended Mitigation:

Unify the inheritance approach so contracts follow a consistent token model. Options include:

1. **Consistent inheritance**: Have all contracts inherit from `AStaticTokenData`, ignoring unused tokens where not needed.
2. **Restructure layers**: Split tokens into clear hierarchies (e.g., CoreTokens for WETH+USDC, ExtendedTokens for LINK and beyond).
3. **Composition over inheritance**: Pass required tokens via constructor parameters instead of through an inheritance chain.

[I-8] Unnecessary contract separation creates architectural complexity

Description:

The protocol unnecessarily separates functionality between `VaultGuardians.sol` and `VaultGuardiansBase.sol`:

- **`VaultGuardians`** inherits from `VaultGuardiansBase` and `Ownable`, but only adds:
  - `updateGuardianStakePrice()` function
  - `updateGuardianAndDaoCut()` function
  - `sweepErc20s()` function (which is fundamentally flawed)
- **`VaultGuardiansBase`** contains all the core protocol logic

This separation adds unnecessary complexity without providing any architectural benefits. The `VaultGuardians` contract serves no purpose beyond being a thin wrapper that could easily be integrated into the base contract.

Impact:

- **Unnecessary complexity**: Two contracts instead of one for no clear reason
- **Maintenance overhead**: Changes require updates in multiple contracts
- **Architectural confusion**: Unclear why the separation exists

Code Location:

```
// src/protocol/VaultGuardians.sol
contract VaultGuardians is Ownable, VaultGuardiansBase {
    // Only adds 3 functions, all of which could be in
       VaultGuardiansBase
}

// src/protocol/VaultGuardiansBase.sol
contract VaultGuardiansBase is AStaticTokenData, IVaultData {
    // Contains all the actual protocol logic
}
```

Recommended Mitigation:

Consolidate the contracts by: 1. **Renaming** `VaultGuardiansBase` to `VaultGuardians` 2. **Making** `VaultGuardians` inherit from `Ownable` directly 3. **Moving** the three functions from the current `VaultGuardians` into the renamed base contract 4. **Removing** the unnecessary `VaultGuardians.sol` file

This simplifies the architecture while maintaining all functionality.

[I-9] Unused import in `InvestableUniverseAdapter.sol`

Description: The interface `IInvestableUniverseAdapter` imports `IERC20`, but it is not used within the interface.

Impact: This is a minor issue of code cleanliness. It can slightly confuse developers reading the code.

Code Location:

```
// src/interfaces/InvestableUniverseAdapter.sol Line: 4
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
```

Recommended Mitigation: Remove the unused import statement.

[I-10] Multiple unused error definitions create code clutter

Description:

Several error definitions in the protocol are never thrown, creating unnecessary code:

1. **`VaultGuardians__TransferFailed`** in `VaultGuardians.sol` - Never used
2. **`VaultGuardiansBase__NotEnoughWeth`** in `VaultGuardiansBase.sol` - Never thrown
3. **`VaultGuardiansBase__CantQuitGuardianWithNonWethVaults`** in `VaultGuardiansBase.sol` - Never thrown
4. **`VaultGuardiansBase__FeeTooSmall`** in `VaultGuardiansBase.sol` - Never thrown

These unused errors suggest incomplete implementation or abandoned features, making the code harder to understand and maintain.

Impact:

- **Code clutter**: Unnecessary error definitions that serve no purpose
- **Maintenance confusion**: Developers may think these errors are used somewhere
- **Audit complexity**: Auditors must determine if these are intentional or oversight
- **Protocol clarity**: Suggests incomplete or evolving design

Code Location:

```
// src/protocol/VaultGuardians.sol
error VaultGuardians__TransferFailed(); // Never used

// src/protocol/VaultGuardiansBase.sol
error VaultGuardiansBase__NotEnoughWeth(uint256 amount, uint256
    amountNeeded); // Never thrown
```

```
6 error VaultGuardiansBase__CantQuitGuardianWithNonWethVaults(address
      guardianAddress); // Never thrown
7 error VaultGuardiansBase__FeeTooSmall(uint256 fee, uint256 requiredFee)
      ; // Never thrown
```

Recommended Mitigation:

Either: 1. **Remove unused errors** if they're not needed 2. **Implement the missing functionality** if these errors represent intended features 3. **Add TODO comments** explaining why these errors exist but are unused

If these errors are meant for future use, document their intended purpose and timeline for implementation. Otherwise, remove them to clean up the codebase.

[I-11] Function ordering violates Solidity style guide recommendations

Description:

The `VaultShares.sol` contract does not follow the recommended Solidity style guide for function ordering. Functions are mixed between public and private visibility, making the code harder to read and maintain.

Current Ordering Issues:

1. **Mixed visibility**: Public functions like `setNotActive()` and `updateHoldingAllocation()` are scattered between private functions like `_investFunds()`
2. **Inconsistent grouping**: The `deposit()` function is placed after private functions, breaking the logical flow

Impact:

- **Reduced readability**: Developers must scan the entire contract to find functions of a specific type
- **Maintenance difficulty**: Code organization doesn't follow established conventions
- **Onboarding confusion**: New developers expect standard Solidity ordering
- **Code review complexity**: Reviewers must mentally reorganize the code structure

Code Location:

```
1 // src/protocol/VaultShares.sol - Current mixed ordering:
2
3 // Public functions scattered throughout:
4 function setNotActive() public onlyVaultGuardians isActive { ... }
5 function updateHoldingAllocation(AllocationData memory
      tokenAllocationData) public onlyVaultGuardians isActive { ... }
```

```
6
7  // Private function mixed in:
8  function _investFunds(uint256 assets) private { ... }
9
10 // Public function after private:
11 function deposit(uint256 assets, address receiver) public override(
       ERC4626, IERC4626) { ... }
12
13 // More private functions:
14 function rebalanceFunds() public isActive divestThenInvest nonReentrant
       {}
15
16 // View functions at the end instead of grouped:
17 function getGuardian() external view returns (address) { ... }
18 function getGuardianAndDaoCut() external view returns (uint256) { ... }
```

Recommended Mitigation:

Reorganize following Solidity style guide (Recommended) Restructure functions in this order and add clear section headers: 1. **Constructor** 2. **Receive/Fallback functions** (if any) 3. **External functions** 4. **Public functions** 5. **Internal functions** 6. **Private functions** 7. **View/Pure functions**

The current ordering makes the contract harder to understand and maintain, especially for developers familiar with Solidity conventions.

[I-12] Interface implementation inconsistency between VaultShares and IVaultShares

Description:

The `VaultShares.sol` contract implements several public and external functions that are not declared in the `IVaultShares.sol` interface, making it incomplete.

Missing Interface Declarations:

- **Investment management**: `rebalanceFunds`
- **View functions**: All getters like `getGuardian`, `getIsActive`, etc.

Note: Core vault functions like `deposit`, `withdraw`, and `redeem` are inherited from `IERC4626` and don't need to be redeclared.

Impact:

- **Incomplete interface**: The implementation doesn't match its declared interface
- **Reduced code clarity**: Developers can't rely on the interface to understand available functions
- **Testing complexity**: Mock contracts based on the interface won't have all necessary functions

Code Location:

```
// src/interfaces/IVaultShares.sol - Only declares:
interface IVaultShares is IERC4626, IVaultData {
    function updateHoldingAllocation(AllocationData memory
        tokenAllocationData) external;
    function setNotActive() external;
    // Missing: deposit, withdraw, redeem, rebalanceFunds, and all
        getters
}

// src/protocol/VaultShares.sol - Implements many more functions:
contract VaultShares is ERC4626, IVaultShares, AaveAdapter,
    UniswapAdapter, ReentrancyGuard {
    // These functions exist but aren't in the interface:
    function rebalanceFunds() public isActive divestThenInvest
        nonReentrant {}
    function getGuardian() external view returns (address) { ... }
    // ... and many more getters
}
```

Recommended Mitigation:

Expand the interface to include all public and external functions.

[I-12] Missing validation for critical protocol parameters

Description:

The `VaultShares.sol` contract lacks validation for several critical parameters that could lead to protocol failures or economic issues:

1. **No validation that `i_guardianAndDaoCut > 0`** - Could cause division by zero in fee calculations
2. **No validation that LP pair exists** - Constructor could set `i_uniswapLiquidityToken` to `address(0)`
3. **No validation that Aave aToken exists** - Constructor could fail silently
4. **No bounds checking on allocation percentages** - Already implemented but worth noting

Impact:

- **Potential crashes**: Division by zero in fee calculations
- **Silent failures**: Protocol could deploy with invalid configurations
- **Economic issues**: Invalid parameters could lead to incorrect fee calculations
- **User experience**: Deposits or withdrawals could fail unexpectedly

Code Location:

```
// src/protocol/VaultShares.sol
constructor(ConstructorData memory constructorData)
    ERC4626(constructorData.asset)
    ERC20(constructorData.vaultName, constructorData.vaultSymbol)
    AaveAdapter(constructorData.aavePool)
    UniswapAdapter(constructorData.uniswapRouter, constructorData.weth,
        constructorData.usdc)
{
    i_guardian = constructorData.guardian;
    i_guardianAndDaoCut = constructorData.guardianAndDaoCut; //  No
       validation > 0
    i_vaultGuardians = constructorData.vaultGuardians;
    s_isActive = true;
    updateHoldingAllocation(constructorData.allocationData);

    // External calls without validation
    i_aaveAToken = IERC20(IPool(constructorData.aavePool)
        .getReserveData(address(constructorData.asset)).aTokenAddress);
    //  No validation that aToken != address(0)

    i_uniswapLiquidityToken = IERC20(i_uniswapFactory
        .getPair(address(constructorData.asset), address(i_weth)));
    //  No validation that pair != address(0)
}
```

Recommended Mitigation:

Add comprehensive parameter validation:

```
constructor(ConstructorData memory constructorData)
    ERC4626(constructorData.asset)
    ERC20(constructorData.vaultName, constructorData.vaultSymbol)
    AaveAdapter(constructorData.aavePool)
    UniswapAdapter(constructorData.uniswapRouter, constructorData.weth,
        constructorData.usdc)
{
    require(constructorData.guardian != address(0), "Invalid guardian")
       ;
    require(constructorData.vaultGuardians != address(0), "Invalid
       vault guardians");
    require(constructorData.guardianAndDaoCut > 0, "Invalid fee cut");

    i_guardian = constructorData.guardian;
    i_guardianAndDaoCut = constructorData.guardianAndDaoCut;
    i_vaultGuardians = constructorData.vaultGuardians;
    s_isActive = true;
    updateHoldingAllocation(constructorData.allocationData);

    // Validate Aave integration
    i_aaveAToken = IERC20(IPool(constructorData.aavePool)
        .getReserveData(address(constructorData.asset)).aTokenAddress);
```

```
20      require(address(i_aaveAToken) != address(0), "Invalid aToken");
21
22      // Validate Uniswap integration
23      i_uniswapLiquidityToken = IERC20(i_uniswapFactory
24          .getPair(address(constructorData.asset), address(i_weth)));
25      require(address(i_uniswapLiquidityToken) != address(0), "Invalid LP
            pair");
26  }
```

Additional Considerations:

1. **Add validation that asset is not zero address**
2. **Validate that allocation percentages sum to exactly 1000**
3. **Add validation that guardian and vault guardians contracts are properly initialized**
4. **Consider adding maximum bounds for fee percentages to prevent excessive fees**

[I-13] Function naming inconsistency in `getUniswapLiquidtyToken()`

Description:

The `getUniswapLiquidtyToken()` function in `VaultShares.sol` has a typo in its name - it should be `getUniswapLiquidityToken()` (missing "i" in "Liquidity").

Impact:

- **Code inconsistency**: Function name doesn't match the variable it returns
- **Developer confusion**: Inconsistent naming makes the code harder to understand
- **Maintenance issues**: Future developers might use the wrong function name

Code Location:

```
1  // src/protocol/VaultShares.sol
2  function getUniswapLiquidtyToken() external view returns (address) { //
       Typo: "Liquidty"
3      return address(i_uniswapLiquidityToken); // Returns "Liquidity"
           token
4  }
```

Recommended Mitigation:

Fix the function name:

```
1  function getUniswapLiquidityToken() external view returns (address) {
       //  Fixed: "Liquidity"
2      return address(i_uniswapLiquidityToken);
3  }
```

Note: This change will break existing integrations that call the function by name, so it should be coordinated with any external systems using this function.

Gas optimizations

[G-1] Using `calldata` instead of `memory` for read-only function parameters

Description: Functions with `external` visibility that take complex types like structs or arrays as read-only arguments can use `calldata` instead of `memory` to save gas. This avoids copying the data from calldata to memory.

Impact: Saves gas on external function calls with large inputs.

Code Location:

```
// src/protocol/VaultGuardiansBase.sol
function becomeGuardian(AllocationData memory wethAllocationData)
    external returns (address) { ... }
function becomeTokenGuardian(AllocationData memory allocationData,
    IERC20 token) external onlyGuardian(i_weth) returns (address) { ...
    }
function updateHoldingAllocation(IERC20 token, AllocationData memory
    tokenAllocationData) external onlyGuardian(token) { ... }

// src/protocol/VaultShares.sol
function updateHoldingAllocation(AllocationData memory
    tokenAllocationData) public onlyVaultGuardians isActive { ... }
```

Recommended Mitigation: Change the data location for `AllocationData` parameters from `memory` to `calldata`. For `updateHoldingAllocation` in `VaultShares.sol`, its visibility must be changed to `external` to use `calldata`.

[G-2] Inefficient storage reads in `_becomeTokenGuardian` function

Description: The `_becomeTokenGuardian` function in `VaultGuardiansBase.sol` performs multiple storage reads (`SLOAD` operations) of the same value `s_guardianStakePrice` that could be optimized by caching the value in memory.

Impact: Each `SLOAD` operation costs 100 gas. The `_becomeTokenGuardian` function reads `s_guardianStakePrice` from storage multiple times, incurring unnecessary gas costs.

Code Location:

```
// src/protocol/VaultGuardiansBase.sol Lines 277-292 - Multiple reads
    of s_guardianStakePrice
```

```
2  function _becomeTokenGuardian(IERC20 token, VaultShares tokenVault)
       private returns (address) {
3      s_guardians[msg.sender][token] = IVaultShares(address(tokenVault));
4      emit GuardianAdded(msg.sender, token);
5      i_vgToken.mint(msg.sender, s_guardianStakePrice);        // 1st
           SLOAD
6      token.safeTransferFrom(msg.sender, address(this),
           s_guardianStakePrice);  // 2nd SLOAD
7      bool succ = token.approve(address(tokenVault), s_guardianStakePrice
           );     // 3rd SLOAD
8      // ... rest of function
9  }
```

Recommended Mitigation: Cache the storage value in a memory variable to avoid multiple SLOAD operations:

```
1  function _becomeTokenGuardian(IERC20 token, VaultShares tokenVault)
       private returns (address) {
2      uint256 stakePrice = s_guardianStakePrice;  // Cache the value once
3
4      s_guardians[msg.sender][token] = IVaultShares(address(tokenVault));
5      emit GuardianAdded(msg.sender, token);
6      i_vgToken.mint(msg.sender, stakePrice);
7      token.safeTransferFrom(msg.sender, address(this), stakePrice);
8      bool succ = token.approve(address(tokenVault), stakePrice);
9      // ... rest of function
10 }
```

This approach saves approximately 300 gas per call by avoiding redundant storage reads of the same value.